

— Databases

Hoofdstuk 7: Subqueries

L. Espeel



— Subqueries ⁽¹⁾

Met een subquery kunnen we eerst informatie opvragen die we nog niet weten om op basis van die informatie een nieuwe query te maken.

Voorbeeld: we willen een overzicht van alle spelers die bij PSG gekomen zijn na de komst van Neymar.

1. We moeten eerst te weten komen wanneer Neymar in dienst is gekomen.
We vragen de *buy_date* op van Neymar:

```
SELECT buy_date
FROM players
WHERE last_name = "da Silva Santos Junior";
```

— Subqueries (2)

2. Daarna nemen we die query en zetten we deze tussen haakjes. Dit wordt de **subquery** of **inner query**.
Hierboven zetten we dan de andere query, dit wordt de **main query** of de **outer query**.

```
SELECT first_name, last_name, buy_date
FROM players
WHERE buy_date >
(
    SELECT buy_date
    FROM players
    WHERE last_name = "da Silva Santos Junior"
);
```

Let op: in een subquery mogen we slechts **één veld selecteren** (hier: *buy_date*). Meerdere velden of * zijn niet toegestaan.
Uitzondering: het kan wel als we de EXISTS-operator gebruiken, zie verder.

3

— Single-record subqueries (1)

Single-record subqueries geven maar één record terug. Ze gebruiken single-record vergelijkingsoperatoren:

```
=
<>
>
>=
<=
```

```
SELECT first_name, last_name, buy_date
FROM players
WHERE buy_date >
(
    SELECT buy_date
    FROM players
    WHERE last_name = "da Silva Santos Junior"
);
```

} De subquery geeft 1 record weer.

4

— Single-record subqueries (2)

Het is mogelijk om ook meerdere single-record subqueries te gebruiken:

```
SELECT last_name, job_id, salary, department_id
FROM employees
WHERE job_id =
(
    SELECT job_id
    FROM employees
    WHERE employee_id = 141
)
AND department_id =
(
    SELECT department_id
    FROM departments
    WHERE location_id = 1500
);
```

De subquery geeft 1 record weer.

De subquery geeft 1 record weer.

5

— Multiple-record subqueries (1)

Multiple-record subqueries kunnen meer dan één resultaat opleveren. Dan kun je de single-record operators (zoals bijvoorbeeld het = teken) niet gebruiken.

Multiple-record vergelijkingsoperatoren:

IN
ANY
ALL

De **NOT** operator kan bij alle drie gebruikt worden.

6

Multiple-record subqueries (2)

IN-operator

Met de **IN-operator** in de outer query kunnen we alle records selecteren die voorkomen in een lijstje van waarden dat gemaakt is met de inner query.

Voorbeeld: stel dat we alle namen van werknemers willen die in hetzelfde jaar zijn aangenomen als werknemers van afdeling 90. Eerst moeten we een subquery (inner query) maken om een lijstje op te halen met de jaren waarin werknemers van afdeling 90 zijn aangenomen.

```
SELECT last_name, YEAR(hire_date)
FROM employees
WHERE YEAR(hire_date) IN
(
  SELECT YEAR(hire_date)
  FROM employees
  WHERE department_id = 90
);
```

De subquery kan meerdere records weergeven.

YEAR(hire_date)
1996
1998
2002

7

hogeschool
vives

Multiple-record subqueries (3)

IN-operator

Multiple-records subqueries zijn altijd "veiliger" dan single-record subqueries. Als je twijfelt tussen IN of =, gebruik dan IN.

```
SELECT first_name, last_name, job_id
FROM employees
WHERE job_id IN
(
  SELECT job_id
  FROM employees
  WHERE last_name = "Ernst"
);
```

```
SELECT first_name, last_name, job_id
FROM employees
WHERE job_id =
(
  SELECT job_id
  FROM employees
  WHERE last_name = "Ernst"
);
```

In dit voorbeeld geeft de subquery 1 record terug, maar ... dit hadden evengoed meerdere records kunnen zijn.

first_name	last_name	job_id
Alexander	Hunold	1
Bruce	Ernst	1
Diana	Lorentz	1

8

hogeschool
vives

Multiple-record subqueries (4)

ANY-operator

Onderstaande query toont het aantal boeken die aangekocht werden na de eerste ontlening in de bibliotheek.

Eerst halen we alle ontleningen uit *tblontleningen*. Daarna vergelijken we of de datum van de aankoop van het boek groter is dan om het even welke ontleningsdatum d.m.v. ANY.

```
SELECT b.titel, b.datum, o.datum_ontlening
FROM tblboeken b
JOIN tblontleningen o ON b.boek_id = o.boek_id
WHERE b.datum > ANY
(
    SELECT datum_ontlening
    FROM tblontleningen
)
ORDER BY b.titel;
```

Ter info, in tabel *tblboeken*:

min. datum = 1900-04-05, max. datum = 2002-10-29

Ter info, in tabel *tblontleningen*:

min datum_ontlening = 1994-07-02, max. datum_ontlening = 2002-10-20

titel	datum	datum_ontlening
Alice in Wonderland	1965-03-09 00:00:00	1999-09-01 00:00:00
Colour of Magic	1999-01-04 00:00:00	2000-04-07 00:00:00
Colour of Magic	1999-01-04 00:00:00	1996-08-04 00:00:00
Colour of Magic	1999-01-04 00:00:00	1994-07-02 00:00:00
Colour of Magic	1999-01-04 00:00:00	1998-09-24 00:00:00
Cujo	1998-01-01 00:00:00	2002-10-20 00:00:00
Cujo	1998-01-01 00:00:00	2000-01-03 00:00:00
David Copperfield	1980-05-09 00:00:00	1999-01-08 00:00:00
David Copperfield	1980-05-09 00:00:00	1999-03-07 00:00:00
De Kappelekensbaan	1965-04-05 00:00:00	2000-04-08 00:00:00
De Val	1982-08-01 00:00:00	2000-06-07 00:00:00
De Val	1982-08-01 00:00:00	2000-02-26 00:00:00
De Val	1982-08-01 00:00:00	2000-03-08 00:00:00
De Val	1982-08-01 00:00:00	2000-01-12 00:00:00

titel	datum	datum_ontlening
Colour of Magic	1999-01-04 00:00:00	1996-08-04 00:00:00
Colour of Magic	1999-01-04 00:00:00	1994-07-02 00:00:00
Colour of Magic	1999-01-04 00:00:00	1998-09-24 00:00:00
Colour of Magic	1999-01-04 00:00:00	2000-04-07 00:00:00
Cujo	1998-01-01 00:00:00	2002-10-20 00:00:00
Cujo	1998-01-01 00:00:00	2000-01-03 00:00:00

9

Multiple-record subqueries (4)

ALL-operator

Onderstaande query toont het aantal boeken die aangekocht werden na de eerste ontlening in de bibliotheek.

Eerst halen we alle ontleningen uit *tblontleningen*. Daarna vergelijken we of de datum van de aankoop van het boek groter is dan elke ontleningsdatum d.m.v. ALL.

```
SELECT b.titel, b.datum, o.datum_ontlening
FROM tblboeken b
JOIN tblontleningen o ON b.boek_id = o.boek_id
WHERE b.datum > ALL
(
    SELECT datum_ontlening
    FROM tblontleningen
)
ORDER BY b.titel;
```

Ter info, in tabel *tblboeken*:

min. datum = 1900-04-05, max. datum = 2002-10-29

Ter info, in tabel *tblontleningen*:

min datum_ontlening = 1994-07-02, max. datum_ontlening = 2002-10-20

titel	datum	datum_ontlening
Alice in Wonderland	1965-03-09 00:00:00	1999-09-01 00:00:00
Colour of Magic	1999-01-04 00:00:00	2000-04-07 00:00:00
Colour of Magic	1999-01-04 00:00:00	1996-08-04 00:00:00
Colour of Magic	1999-01-04 00:00:00	1994-07-02 00:00:00
Colour of Magic	1999-01-04 00:00:00	1998-09-24 00:00:00
Cujo	1998-01-01 00:00:00	2002-10-20 00:00:00
Cujo	1998-01-01 00:00:00	2000-01-03 00:00:00
David Copperfield	1980-05-09 00:00:00	1999-01-08 00:00:00
David Copperfield	1980-05-09 00:00:00	1999-03-07 00:00:00
De Kappelekensbaan	1965-04-05 00:00:00	2000-04-08 00:00:00
De Val	1982-08-01 00:00:00	2000-06-07 00:00:00
De Val	1982-08-01 00:00:00	2000-02-26 00:00:00
De Val	1982-08-01 00:00:00	2000-03-08 00:00:00
De Val	1982-08-01 00:00:00	2000-01-12 00:00:00

titel	datum	datum_ontlening
-------	-------	-----------------

10

— Exists en not exists (1)

Stel dat we willen onderzoeken welke *department_id*'s in de tabel *departments* staan en ook bestaan in de tabel *employees*.

```
SELECT d.department_id, d.department_name
FROM departments d
WHERE EXISTS
(
  SELECT *
  FROM employees e
  WHERE d.department_id = e.department_id
)
ORDER BY d.department_id;
```

department_id	department_name
10	Finance
20	HR
30	IT

employee_id	name	department_id
1	Anna	10
2	Boris	20
3	Carla	30
4	David	30

Opmerking: de EXISTS-operator ondersteunt subqueries waarin we meerdere velden selecteren.

11

— Exists en not exists (2)

Stel dat we willen onderzoeken welke *department_id*'s niet voorkomt in *employees*.

```
SELECT d.department_id, d.department_name
FROM departments d
WHERE NOT EXISTS
(
  SELECT *
  FROM employees e
  WHERE d.department_id = e.department_id
)
ORDER BY d.department_id;
```

department_id	department_name
40	Sales
50	Facilities

employee_id	name	department_id
1	Anna	10
2	Boris	20
3	Carla	30
4	David	30

12

— Subqueries met statistische functies

We kunnen ook statistische functies gebruiken in subqueries (COUNT, MIN, MAX, SUM en AVG). Zo kunnen we berekenen wie er bv. minder verdient dan het gemiddeld salaris.

```
SELECT last_name, salary
FROM employees
WHERE salary <
(
  SELECT AVG(salary)
  FROM employees
);
```

} Geeft een getalwaarde terug.